

```

// main.cpp

#define NODEMCU_FILE

#define NOTIFY_INTERVAL_MS 60000 // 1 minute

unsigned long last_notify = 0;

#include <Arduino.h>
#include <SoftwareSerial.h>

#include ".common/comm.h"
#include "CsvOperations.h"
#include "ESP8266WebServer.h"
#include "SetupFunctions.h"
#include "WebServer.h"

// opting for .csv because it's faster to parse and write
// than a formatted json file

#define EXPIRY_INTERVL 3000011

#define HTTP_PORT 80
#define DAYS_GOING_BAD 2
// for perishables we cannot monitor, we broadly
// assume that the item will expire ~2 days before
// the final expiry date, except for items that items
// expires in two days, somehow (???).

#define MAX_BOX_COUNT 8
// Current Testing Value, total count should be ~260 boxes

SoftwareSerial arduino_wire(D2, D1);
ESP8266WebServer server(80);

unsigned long time_tc = 0;

void setup()
{
    d_SerialBegin(9600);

    WifiSetup();
    NTPSetup();
    NTFYSetup();
    WebSetup(&server);
    SDSetup();
    BootNotify();

    arduino_wire.begin(9600);
}

void loop()
{

```

```
unsigned long rn = millis();
server.handleClient();
MessageHandler(&arduino_wire);

if (rn - time_tc >= EXPIRY_INTERVL)
{
    time_tc = rn;
    CheckAndNotify();
}
}
```

```

// SetupFunctions.h

#include "SoftwareSerial.h"
#define NODEMCU_FILE

#include <stdint.h>

/** @file SetupFunctions.h
 * @brief Sets Up WiFi and other networking features
 */

#pragma once

#define HTTP_PORT 80

/**
 * @brief Sets up the WiFi on the NodeMCU.
 *
 * @details Iterates through "pass_list" (defined in sensitive_data.c)
 *
 * @note It only goes to the first successful connection, and not best
 *       connection, as in the current build the webserver's size isn't
 *       really known yet. [09 Mar 2026]
 */
void WifiSetup();

/**
 * @brief Sets up the connection to the NTP time pool.
 *
 * @note We need to do this, since there was an initial bug where new
 *       logged items where set to be registered from the unix time epoch
 */
void NTPSetup();

/**
 * @brief Sets up the connection to the ntfy.sh topic.
 */
void NTFYSetup();

/**
 * @brief Ensures that the inventory file, and all related files are available on the SD
card
 *       at the time of boot.
 */
void SDSSetup();

/**
 * @brief Entry notification that is sent on the booting of the system.
 */
void BootNotify();

/**
 * @brief Handles all the communication with the Arduino Uno unit.

```

```

*/
void MessageHandler(SoftwareSerial *serial);

// SetupFunctions.cpp

#include "SetupFunctions.h"

#include "CSV_Parser.h"
#include "SoftwareSerial.h"
#include "Updater.h"
#include "nodemcu/CsvOperations.h"
#include "nodemcu/NtfyNotifier.h"
#include "wl_definitions.h"
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <IPAddress.h>

#include ".common/comm.h"
#include <NTPClient.h>
#include <WiFiUdp.h>

#include <SD.h>

#include "sensitive_data.cpp"
// Contains defines for:
//      -> NTFY_SERVER
//      -> NTFY_PORT
//      -> NTFY_TOPIC
//      -> pass_list: Array of wifi_pass_t (as defined in .common/comm.h)

#include "SetupFunctions.h"

WiFiUDP ntp_udp;
NTPClient time_client(ntp_udp, "pool.ntp.org", 0, 0);

void WifiSetup()
{
    for (wifi_pass_t wifi_pass : pass_list)
    {
        d_SerialPrint("Connecting to...");
        d_SerialPrintV(wifi_pass.ssid);

        WiFi.begin(wifi_pass.ssid, wifi_pass.pass);

        while ((WiFi.status() != WL_CONNECTED) and (WiFi.status() != WL_WRONG_PASSWORD))
        {
            d_delay(500);
            d_SerialPrint(".");
        }

        switch (WiFi.status()) // Using a switch so that other cases can be handled in the
        future
        {

```

```

    case WL_CONNECTED:
        d_SerialPrintln("\nWiFi connected");
        d_SerialPrint("IP address: ");
        d_SerialPrintlnV(WiFi.localIP());
        return;
        // Kill the function on connecting
    case WL_WRONG_PASSWORD:
        d_SerialPrintf("\nWrong Password for \"%s\"", wifi_pass.ssid);
        continue;
    default:
        continue;
}
}
}

```

```

void NTPSetup()
{
    time_client.begin();

    d_SerialPrintln("Waiting for NTP...");
    while (!time_client.update())
    {
        d_SerialPrint(".");
        delay(500);
    }

    time_t epochTime = time_client.getEpochTime();
    timeval tv = {epochTime, 0};
    settimeofday(&tv, nullptr);
    d_SerialPrintf("Current Unix time: %lld\n", time(nullptr));
}

```

```

void NTFYSetup()
{
    IPAddress ntfy_ip;

    d_SerialPrintf("Resolving host :%s\n", NTFY_SERVER);
    if (WiFi.hostByName(NTFY_SERVER, ntfy_ip) == 0)
    {
        d_SerialPrintln("Fatal: DNS Resolution failed for ntfy.sh!");
        return;
    }
}

```

```

void SDSSetup()
{
    time_client.update();

    if (!SD.begin(D8))
    {
        d_SerialPrintln("SD Init Failed!");
        return;
    }
}

```

```

d_SerialPrintln("SD Card Init'ized.");

if (!SD.exists(INVENTORY_FILE))
{
    d_SerialPrintln("File not found! Creating...");
    if (File f = SD.open(INVENTORY_FILE, FILE_WRITE))
    {
        f.println(""); // Since we're ditching headers now.
        f.close();
    }
    d_SerialPrintln("File Created.");
}
else
    d_SerialPrintln("File Exists. Continuing...");

if (SD.exists(INVENTORY_FILE_TEMP))
    SD.remove(INVENTORY_FILE_TEMP);
}

void MessageHandler(SoftwareSerial *serial)
{
    static char buffer[128];
    static size_t idx = 0;
    static bool waiting_for_csv = false;
    static unsigned long last_byte_time = 0;
    while (serial->available())
    {
        const char c = serial->read();
        last_byte_time = millis();
        if (c == MSG_TERMINATOR)
        {
            if (idx == 0)
                continue;
            buffer[idx] = '\0';
            d_SerialPrint("RX: ");
            d_SerialPrintln(buffer);
            if (waiting_for_csv)
            {
                UpdateCSV(buffer);
                waiting_for_csv = false;
            }
            else if (strcmp(buffer, ReqActBox) == 0)
            {
                d_SerialPrintln("RAB received");
                WritePhysicals(serial);
            }
            else if (strcmp(buffer, UpdBoxes) == 0)
            {
                d_SerialPrintln("UPD received, sending OK");
                serial->print("OK");
                serial->print(MSG_TERMINATOR);
                waiting_for_csv = true;
            }
        }
    }
}

```

```

}
else if (strcmp(buffer, "IVC") == 0)
{
    d_SerialPrintln("IVC received");
    uint16_t count = InventoryCount();
    serial->print("IVC,");
    serial->print(count);
    serial->print(MSG_TERMINATOR);
    d_SerialPrint("IVC Sent back: IVC,");
    d_SerialPrintln(count);
}
else if (strncmp(buffer, "IVW,", 4) == 0)
{
    d_SerialPrintln("IVW received");
    char *save = nullptr;
    char *start_tok = strtok_r(buffer + 4, ",", &save);
    char *rows_tok = strtok_r(nullptr, ",", &save);
    uint16_t start = start_tok ? (uint16_t)atoi(start_tok) : 0;
    uint8_t rows = rows_tok ? (uint8_t)atoi(rows_tok) : 2;
    if (rows == 0)
        rows = 2;
    WriteInventoryWindow(serial, start, rows);
}
else
{
    d_SerialPrintln("Unknown message");
}
idx = 0;
}
else
{
    if (idx < sizeof(buffer) - 1)
        buffer[idx++] = c;
    else
        idx = 0;
}
}
if (idx > 0 && millis() - last_byte_time > 200)
    idx = 0;
}

```

```

// CsvOperations.h

#pragma once

#define NODEMCU_FILE

#define INVENTORY_FILE_TEMP "/temp.csv"

#define BUFFER_LIMIT 64
// The total buffer size of a completely
// packed csv line would actually be 48
// bytes, but 64 allows a safe margin and
// is a round number

#include ".common/comm.h"
#include "SoftwareSerial.h"
#include <CSV_Parser.h>
#include <stdint.h>

#include <SD.h>
#include <SPI.h>

struct InventoryRecordLine
{
    uint32_t reg_date;
    uint32_t expiry_date;
    char item_name[16];
    char box_id[3];
    uint8_t notified;
    bool physical;
    bool supported;
    FoodCondition condition;
    ProfileID preset;
};

void WritePhysicals(SoftwareSerial *sender);
void SaveInventory(InventoryRecordLine *records);
void AddRecord(const InventoryRecordLine *record);
void DeleteRecord(const char *target_box_id);
void CleanCSV();
void UpdateCSV(const char *csv_input);
void CheckAndNotify();

uint16_t InventoryCount();
void WriteInventoryWindow(SoftwareSerial *sender, uint16_t start, uint8_t count);

// CsvOperations.cpp

#include "CsvOperations.h"

#include "NtfyNotifier.h"
#include <Arduino.h>
#include <CSV_Parser.h>

```

```

#include <SD.h>
#include <SPI.h>
#include <SoftwareSerial.h>
#include <cstdint>

#include ".common/comm.h"
#include "WString.h"
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <sys/types.h>

File read_file;
File edit_file;
File copy_file;

inline void InventoryInit()
{
    if (read_file)
        read_file.close();

    if ((read_file = SD.open(INVENTORY_FILE, FILE_READ)))
        d_SerialPrintln("File Exists.");
    else
        d_SerialPrintln("Error! File not found!");
}

// we're going to be using row-by-row indexing to fix memory/buffer overflows
// order is something like this
// 0          1          2          3          4          5          6
// notified, box_id, item_name, reg_date, expiry_date, physical, condition
// uint8_t,  string, string,      ulong,      ulong,          uint8_t,  uint8_t

void WritePhysicals(SoftwareSerial *sender)
{
    InventoryInit();

    char field[32];
    uint8_t field_idx = 0;
    uint8_t col = 0;

    char box_id[16] = {0};
    uint16_t profile = 0;
    bool is_physical = false;

    while (read_file.available())
    {
        char c = read_file.read();

        if (c == ',' || c == '\n' || c == '\r')
        {
            field[field_idx] = '\0';

            if (col == 1)

```

```

    {
        // box_id
        strncpy(box_id, field, sizeof(box_id) - 1);
        box_id[sizeof(box_id) - 1] = '\0';
    }
    else if (col == 5)
        // physical flag
        is_physical = (field[0] == '1');
    else if (col == 7)
        // profile enum
        profile = (uint16_t)atoi(field);

    field_idx = 0;
    col++;

    if (c == '\n' || c == '\r')
    {
        if (is_physical)
        {
            sender->print(box_id);
            sender->print(',');
            sender->print(profile);
            sender->print(';');
        }

        col = 0;
        profile = 0;
        is_physical = false;
        box_id[0] = '\0';
    }
}
else
{
    if (field_idx < sizeof(field) - 1)
        field[field_idx++] = c;
}
}

read_file.close();

sender->print(MSG_TERMINATOR);
}

void AddRecord(const InventoryRecordLine *record)
{
    d_SerialPrint("Free Heap:");
    d_SerialPrintlnV(ESP.getFreeHeap());

    if ((edit_file = SD.open(INVENTORY_FILE, FILE_WRITE)))
    {
        edit_file.printf("0,%s,%s,%u,%u,%d,%d,%u\n", record->box_id, record->item_name,
record->reg_date,
                        record->expiry_date, record->physical, record->condition, record->

```

```

>preset);
    edit_file.close();
    return;
}
d_SerialPrint("Error! File Not found");
}

void DeleteRecord(const char *target_box_id)
{
    File in_file = SD.open(INVENTORY_FILE, FILE_READ);
    if (!in_file)
        return;

    File out_file = SD.open(INVENTORY_FILE_TEMP, FILE_WRITE);
    if (!out_file)
    {
        in_file.close();
        return;
    }

    char line_buffer[BUFFER_LIMIT];

    while (in_file.available())
    {
        size_t len = in_file.readBytesUntil('\n', line_buffer, sizeof(line_buffer) - 1);
        line_buffer[len] = '\0';

        while (len > 0 && (line_buffer[len - 1] == '\n' || line_buffer[len - 1] == '\r'))
            line_buffer[--len] = '\0';

        char *cols[8];
        char *token = strtok(line_buffer, ",");

        int i = 0;
        while (token && i < 8)
        {
            cols[i++] = token;
            token = strtok(NULL, ",");
        }

        if (i < 8)
            continue;

        if (strcmp(cols[1], target_box_id) == 0)
            continue;

        out_file.print(cols[0]); // notified
        out_file.print(',');
        out_file.print(cols[1]); // box_id
        out_file.print(',');
        out_file.print(cols[2]); // item_name
        out_file.print(',');
        out_file.print(cols[3]); // reg_date
    }
}

```

```

    out_file.print(',');
    out_file.print(cols[4]); // expiry_date
    out_file.print(',');
    out_file.print(cols[5]); // physical
    out_file.print(',');
    out_file.print(cols[6]); // condition
    out_file.print(',');
    out_file.println(cols[7]); // profile
}

in_file.close();
out_file.close();

SD.remove(INVENTORY_FILE);
SD.rename(INVENTORY_FILE_TEMP, INVENTORY_FILE);
}

void UpdateCSV(const char *csv_input)
{
    const int max_updates = 16;

    char update_ids[max_updates][10];
    uint8_t update_conditions[max_updates];
    int update_count = 0;

    char buffer[BUFFER_LIMIT];
    strncpy(buffer, csv_input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    char *line = buffer;

    while (*line != '\0' && update_count < max_updates)
    {
        char *next_line = strchr(line, '\n');
        if (next_line)
        {
            *next_line = '\0';
            next_line++;
        }

        size_t line_len = strlen(line);
        while (line_len > 0 && (line[line_len - 1] == '\r' || line[line_len - 1] == '\n'))
            line[--line_len] = '\0';

        if (*line != '\0')
        {
            char *comma = strchr(line, ',');
            if (comma)
            {
                *comma = '\0';
                char *id = line;
                char *cond = comma + 1;
            }
        }
    }
}

```

```

        strncpy(update_ids[update_count], id, sizeof(update_ids[0]) - 1);
        update_ids[update_count][sizeof(update_ids[0]) - 1] = '\0';
        update_conditions[update_count] = (uint8_t)atoi(cond);
        update_count++;
    }
}

if (!next_line)
    break;

line = next_line;
}

File in_file = SD.open(INVENTORY_FILE, FILE_READ);
if (!in_file)
    return;

File out_file = SD.open(INVENTORY_FILE_TEMP, FILE_WRITE);
if (!out_file)
{
    in_file.close();
    return;
}

char line_buffer[BUFFER_LIMIT];

while (in_file.available())
{
    size_t len = in_file.readBytesUntil('\n', line_buffer, sizeof(line_buffer) - 1);
    line_buffer[len] = '\0';

    while (len > 0 && (line_buffer[len - 1] == '\n' || line_buffer[len - 1] == '\r'))
        line_buffer[--len] = '\0';

    char *cols[8];
    char *token = strtok(line_buffer, ",");
    int i = 0;

    while (token && i < 8)
    {
        cols[i++] = token;
        token = strtok(NULL, ",");
    }

    if (i < 8)
        continue;

    uint8_t notified = (uint8_t)atoi(cols[0]);
    uint8_t current_condition = (uint8_t)atoi(cols[6]);

    for (int j = 0; j < update_count; j++)
    {
        if (strcmp(cols[1], update_ids[j]) == 0)

```

```

    {
        uint8_t incoming_condition = update_conditions[j];
        bool should_update = false;

        if (current_condition == UNKNOWN && incoming_condition != UNKNOWN)
        {
            should_update = true;
        }
        else if (incoming_condition > current_condition)
        {
            should_update = true;
        }

        if (should_update)
        {
            current_condition = incoming_condition;
            notified = 0;
        }

        break;
    }
}

out_file.print(notified);
out_file.print(',');
out_file.print(cols[1]); // box_id
out_file.print(',');
out_file.print(cols[2]); // item_name
out_file.print(',');
out_file.print(cols[3]); // reg_date
out_file.print(',');
out_file.print(cols[4]); // expiry_date
out_file.print(',');
out_file.print(cols[5]); // physical
out_file.print(',');
out_file.print(current_condition);
out_file.print(',');
out_file.println(cols[7]); // profile
}

in_file.close();
out_file.close();

SD.remove(INVENTORY_FILE);
SD.rename(INVENTORY_FILE_TEMP, INVENTORY_FILE);
}

```

```

FoodCondition WorseCondition(FoodCondition a, FoodCondition b)
{
    if (a == UNKNOWN)
        return b;
    if (b == UNKNOWN)
        return a;
}

```

```

    return (a > b) ? a : b;
}

void CheckAndNotify()
{
    File in_file = SD.open(INVENTORY_FILE, FILE_READ);
    if (!in_file)
        return;

    SD.remove(INVENTORY_FILE_TEMP);
    File out_file = SD.open(INVENTORY_FILE_TEMP, FILE_WRITE);
    if (!out_file)
    {
        in_file.close();
        return;
    }

    char line_buffer[BUFFER_LIMIT];
    uint32_t now = (uint32_t)time(nullptr);

    while (in_file.available())
    {
        size_t len = in_file.readBytesUntil('\n', line_buffer, sizeof(line_buffer) - 1);
        line_buffer[len] = '\0';

        while (len > 0 && (line_buffer[len - 1] == '\n' || line_buffer[len - 1] == '\r'))
            line_buffer[--len] = '\0';

        char *cols[8];
        char *token = strtok(line_buffer, ",");
        int i = 0;

        while (token && i < 8)
        {
            cols[i++] = token;
            token = strtok(NULL, ",");
        }

        if (i < 8)
            continue;

        uint8_t notified = (uint8_t)atoi(cols[0]);
        const char *box_index = cols[1];
        const char *item_name = cols[2];
        uint32_t reg_date = (uint32_t)atol(cols[3]);
        uint32_t expiry = (uint32_t)atol(cols[4]);
        bool physical = atoi(cols[5]) != 0;
        FoodCondition stored_condition = (FoodCondition)atoi(cols[6]);
        const char *preset = cols[7];

        FoodCondition time_condition = FRESH;

        if (expiry <= now)

```

```

{
    time_condition = EXPIRED;
}
else if (expiry > reg_date && now >= reg_date && now <= expiry)
{
    uint32_t total_life = expiry - reg_date;
    uint32_t remaining_life = expiry - now;

    if (total_life > 0)
    {
        uint32_t percent_remaining = (uint32_t)((((uint64_t)remaining_life * 100ULL) /
(uint64_t)total_life));

        if (percent_remaining <= 30)
            time_condition = GOING_BAD;
    }
}

FoodCondition final_condition = WorseCondition(stored_condition, time_condition);

if (final_condition > stored_condition)
    notified = 0;

if (notified == 0 && final_condition != UNKNOWN && final_condition != FRESH)
{
    switch (final_condition)
    {
        case GOING_BAD:
        {
            String msg = String("*") + item_name + "* in box *" + box_index + "* is
going bad.";
            NtfyNotify("warning", "Item Going Bad", msg.c_str(), "high");
            notified = 1;
        }
        break;

        case EXPIRED:
        {
            String msg = String("*") + item_name + "* in box *" + box_index + "* has
expired!";
            NtfyNotify("warning,skull", "Item Expired!", msg.c_str(), "urgent");
            notified = 1;
        }
        break;

        default:
            break;
    }
}

out_file.print(notified);
out_file.print(',');
out_file.print(box_index);

```

```

    out_file.print(',');
    out_file.print(item_name);
    out_file.print(',');
    out_file.print(reg_date);
    out_file.print(',');
    out_file.print(expiry);
    out_file.print(',');
    out_file.print(physical ? 1 : 0);
    out_file.print(',');
    out_file.print((uint8_t)final_condition);
    out_file.print(',');
    out_file.println(preset);
}

in_file.close();
out_file.close();

SD.remove(INVENTORY_FILE);
SD.rename(INVENTORY_FILE_TEMP, INVENTORY_FILE);
}

uint16_t InventoryCount()
{
    d_SerialPrintln("InventoryCount: begin");

    InventoryInit();

    if (!read_file)
    {
        d_SerialPrintln("InventoryCount: read_file invalid");
        return 0;
    }

    uint16_t count = 0;
    bool has_data = false;

    while (read_file.available())
    {
        char c = read_file.read();

        if (c != '\n' && c != '\r')
            has_data = true;

        if ((c == '\n' || c == '\r') && has_data)
        {
            count++;
            has_data = false;

            if (c == '\r' && read_file.available())
            {
                char next = read_file.peek();
                if (next == '\n')
                    read_file.read();
            }
        }
    }
}

```

```

    }
}

if (has_data)
    count++;

read_file.close();

d_SerialPrint("InventoryCount: final count = ");
d_SerialPrintln(count);

return count;
}

void WriteInventoryWindow(SoftwareSerial *sender, uint16_t start, uint8_t rows)
{
    InventoryInit();
    if (!read_file)
    {
        sender->print("IVW,");
        sender->print(MSG_TERMINATOR);
        return;
    }

    sender->print("IVW,");

    char field[40];
    uint8_t field_idx = 0;
    uint8_t col = 0;

    char item_name[32] = {0};
    char box_id[16] = {0};
    char condition[8] = "3";

    uint16_t row_index = 0;
    uint8_t sent = 0;
    bool first_row = true;

    while (read_file.available() && sent < rows)
    {
        char c = read_file.read();

        if (c == ',' || c == '\n' || c == '\r')
        {
            field[field_idx] = '\0';

            if (col == 1)
            {
                strncpy(box_id, field, sizeof(box_id) - 1);
                box_id[sizeof(box_id) - 1] = '\0';
            }
            else if (col == 2)

```

```

{
    strncpy(item_name, field, sizeof(item_name) - 1);
    item_name[sizeof(item_name) - 1] = '\0';
}
else if (col == 6)
{
    strncpy(condition, field, sizeof(condition) - 1);
    condition[sizeof(condition) - 1] = '\0';
}

field_idx = 0;
col++;

if (c == '\n' || c == '\r')
{
    if (col > 1)
    {
        if (row_index >= start)
        {
            if (!first_row)
                sender->print('\n');

            sender->print(item_name);
            sender->print('|');
            sender->print(condition);
            sender->print('|');
            sender->print(box_id);

            first_row = false;
            sent++;
        }

        row_index++;
    }

    col = 0;
    item_name[0] = '\0';
    box_id[0] = '\0';
    strcpy(condition, "3");

    if (c == '\r' && read_file.available())
    {
        char next = read_file.peek();
        if (next == '\n')
            read_file.read();
    }
}
else
{
    if (field_idx < sizeof(field) - 1)
        field[field_idx++] = c;
}

```

```
}
```

```
read_file.close();
```

```
sender->print(MSG_TERMINATOR);
```

```
}
```

```

// NtfyNotifier.h

#define NODEMCU_FILE

#pragma once

#define PRIORITY_DEFAULT "3"
#define PRIORITY_URGENT "5"

#include "CsvOperations.h"

void NtfyNotify(const char *tags, const char *title, const char *message, const char
*priority);
void NotifyItemAdded(const InventoryRecordLine *record);
void NotifyItemDeleted(const char *box_id);

// NtfyNotifier.cpp

#include "NtfyNotifier.h"

#include "CsvOperations.h"
#include "ESP8266HTTPClient.h"
#include "IPAddress.h"
#include "WString.h"
#include "WiFiClient.h"

#include ".common/comm.h"
#include "sensitive_data.cpp"

void NtfyNotify(const char *tags, const char *title, const char *message, const char
*priority)
{
    const String topicPath = String("/") + String(NTFY_TOPIC);

    HTTPClient http;
    IPAddress ntfy_ip;
    WiFiClient esp_client;

    if (!http.begin(esp_client, String(NTFY_SERVER), NTFY_PORT, topicPath))
    {
        d_SerialPrintln("Fatal: HTTP connect failed.");
        http.end();
        return;
    }

    http.addHeader("X-Title", title);
    http.addHeader("X-Priority", priority);
    http.addHeader("X-Tags", tags);
    http.addHeader("Markdown", "yes");

    int http_response = http.POST(message);

    if (http_response > 0)

```

```

        d_SerialPrintf("NTFY Post success. Code %d\n", http_response);
    else
        d_SerialPrintf("NTFY Post Failed. Error %s(%d)\n",
http.errorToString(http_response).c_str(), http_response);

    http.end();
    yield();
}

void NotifyItemAdded(const InventoryRecordLine *record)
{
    String msg = String("**") + record->item_name + "** has been added to box **" +
record->box_id + "**.";
    NtfyNotify("white_check_mark,package", "Item Added", msg.c_str(), "default");
}

void NotifyItemDeleted(const char *box_id)
{
    String msg = String("Item in box **") + box_id + "** has been deleted from
inventory.";
    NtfyNotify("wastebasket", "Item Deleted", msg.c_str(), "default");
}

void BootNotify()
{
    time_t now = time(nullptr);
    struct tm *t = localtime(&now);
    char time_str[32];
    strftime(time_str, sizeof(time_str), "%d/%m/%Y %H:%M:%S", t);
    String msg = String("System booted successfully at **") + time_str + "**. Ready to
monitor inventory.";
    NtfyNotify("white_check_mark,rocket", "System Ready", msg.c_str(), "default");
}

```

```
// WebServer.h
```

```
#define NODEMCU_FILE
```

```
#include <ESP8266WebServer.h>
```

```
void handleRoot(ESP8266WebServer *server);  
void handleGetInventory(ESP8266WebServer *server);  
void handleAdd(ESP8266WebServer *server);  
void handleDelete(ESP8266WebServer *server);  
void handleClear(ESP8266WebServer *server);  
void WebSetup(ESP8266WebServer *server);
```

```
// WebServer.cpp
```

```
#include "WebServer.h"
```

```
#include "CsvOperations.h"
```

```
#include "NtfyNotifier.h"
```

```
#include ".common/comm.h"
```

```
#include <ESP8266WebServer.h>
```

```
#include <ESP8266mDNS.h>
```

```
#include <SD.h>
```

```
void WebSetup(ESP8266WebServer *server)
```

```
{
```

```
    if (MDNS.begin("tim")) // Why does this stupid thign add  
                           // 2KB of overhead....whyyyy
```

```
        d_SerialPrintln("mDNS responder start!");
```

```
    else
```

```
        d_SerialPrintln("mDNS startup failed!");
```

```
    server->on("/", HTTP_GET, [server]() { handleRoot(server); });
```

```
    server->on("/inventory", HTTP_GET, [server]() { handleGetInventory(server); });
```

```
    server->on("/add", HTTP_POST, [server]() { handleAdd(server); });
```

```
    server->on("/delete", HTTP_POST, [server]() { handleDelete(server); });
```

```
    server->on("/clear", HTTP_POST, [server]() { handleClear(server); });
```

```
    server->begin();
```

```
    d_SerialPrintln("Webserver started!");
```

```
}
```

```
void handleClear(ESP8266WebServer *server)
```

```
{
```

```
    if (SD.exists(INVENTORY_FILE))
```

```
        SD.remove(INVENTORY_FILE);
```

```
    File file = SD.open(INVENTORY_FILE, FILE_WRITE);
```

```
    if (!file)
```

```
    {
```

```
        server->send(500, "text/plain", "Failed to recreate file");
```

```
        return;
```

```

    }

    file.close();
    server->send(200, "text/plain", "Cleared");
}

void handleRoot(ESP8266WebServer *server)
{
    File f = SD.open("/index.html", FILE_READ);
    if (!f)
    {
        server->send(404, "text/plain", "index not found!");
        return;
    }
    server->streamFile(f, "text/html");

    f.close();
}

void handleGetInventory(ESP8266WebServer *server)
{
    File f = SD.open(INVENTORY_FILE, FILE_READ);
    if (!f)
    {
        server->send(500, "text/plain", "File not found");
        return;
    }
    server->streamFile(f, "text/csv");
    f.close();
}

void handleAdd(ESP8266WebServer *server)
{
    if (!server->hasArg("box_id") or !server->hasArg("item") or !server->hasArg("expiry")
or
        !server->hasArg("physical") or !server->hasArg("profile"))
    {
        server->send(400, "text/plain", "Missing fields");
        return;
    }

    InventoryRecordLine record;

    strncpy(record.box_id, server->arg("box_id").c_str(), sizeof(record.box_id));
    strncpy(record.item_name, server->arg("item").c_str(), sizeof(record.item_name));
    record.reg_date = time(nullptr);
    record.expiry_date = server->arg("expiry").toInt();
    record.physical = server->arg("physical") == "1";
    record.condition = FoodCondition::FRESH;
    record.notified = 0;
    record.preset = (ProfileID)(server->arg("profile").toInt());

    AddRecord(&record);
}

```

```
    NotifyItemAdded(&record);

    server->send(200, "text/plain", "Ok.");
}

void handleDelete(ESP8266WebServer *server)
{
    if (!server->hasArg("box_id"))
    {
        server->send(400, "text/plain", "Missing box_id");
        return;
    }
    DeleteRecord(server->arg("box_id").c_str());
    NotifyItemDeleted(server->arg("box_id").c_str());
    server->send(200, "text/plain", "Ok.");
}
```